

Part A

1.1

Rest stand for “Representational State Transfer.”

1.2

A “resource” is any piece of information or object that can be named, addressed, and manipulated through the API. It is a conceptual mapping to a set of entities (e.g., a student, an order, a product). In REST, resources are identified by URLs (e.g., `/api/students` represents the collection of all students, and `/api/students/5` represents a single student with ID 5). Resources are typically represented in a format such as JSON or XML when transferred between client and server.

1.3

This URL represents the collection resource of all students. It is the endpoint used to interact with the entire set of student records.

1.4

This URL represents a single, specific student resource. The student whose ID is 10. It is used to retrieve, update, or delete that particular student. For example, GET `/api/students/10` returns the details of student 10, PUT updates that student, and DELETE removes that student.

1.5

RESTful URLs should use **nouns** because the URL should identify the **resource** (the thing), not the **action** being performed on it. The action is already expressed by the HTTP method (GET, POST, PUT, DELETE). For example:

- Good: GET `/api/students` (the noun "students" identifies the resource; GET expresses the action)
- Bad: GET `/api/getStudents` (the URL contains an action verb)

Using nouns keeps URLs clean, consistent, and predictable. It also avoids redundancy and makes the API more intuitive to use, since the same URL can support multiple operations depending on the HTTP method used.

1.6

REST is **stateless**, meaning that each request from a client to the server must contain all the information needed for the server to understand and process the request. The server does not store any client context or session state between requests. Every request is independent and self-contained.

For example, if a client wants to retrieve a specific student, it must include the student's ID in the URL (/api/students/10) — the server does not remember that the client previously asked for student 10. This improves scalability, reliability, and visibility, because any server can handle any request without needing to know what happened before.

1.7

It is easy to read/write for both humans and machines. It is **language-independent** but maps naturally to data structures in most programming languages (objects, arrays, key-value pairs). It has a compact syntax compared to XML, reducing payload size and improving performance. It is easy to parse **and generate** in [ASP.NET](#) Core, which automatically serializes C# objects to JSON.

Part B

Requirement	HTTP Method	URL
Retrieve all students	GET	/api/students
Retrieve student 5	GET	/api/students/5
Add a new student	POST	/api/students
Update student 5	PUT	/api/students/5
Delete student 5	DELETE	/api/students/5

Part C

Field	C# Data type	Reason
ID	int	Integer is standard for ID
Name	string	Text value
Surname	string	Text value
Email	string	Text value
Course	string	Text value
Year	int	Numeric value

Part M

Test	Method	URL	Expected Status
Get all students	GET	/api/students	200
Get student	GET	/api/students/1	200
Get non-existent student	GET	/api/students/999	404
Create student	POST	/api/students	201
Update student	PUT	/api/students/1	200
Update non-existent student	PUT	/api/students/999	404
Delete student	DELETE	/api/students/1	204
Delete non-existent student	DELETE	/api/students/999	404

Part N

URL	RESTful	Reason
/api/students	Yes	Noun-based, identifies the collection resource
/api/getStudents	No	Contains action verb
/api/students/5	Yes	Noun based, identifies a specific resource by ID
/api/createStudents	No	Contains an action verb
/api/students/delete/5	No	Contains an action verb
/api/students/5	Yes	Noun-based
/api/students/5	Yes	Noun_based

Part O

Status Code	Situation
200	Successful GET
201	Student successfully created
204	Successful DELETE
400	Invalid request
404	Student does not exist
500	Server-side error

Additional codes mentioned:

401 Unauthorized- Authentication is required and it was failed to be provided

403 Forbidden- Server understood request, but refuses to authorize it.

Part Q

MVC	REST API
Returns HTML views	Returns data
Uses return View()	Uses “return OK(),” “return NotFound()”
Designed around user interfaces and pages	Designed around resources and HTTP methods
Browser-oriented	Client/app-oriented
Typically uses sessions and cookies	Stateless

An ASP.NET Core app can contain both MVC controllers and API controllers because the framework is modular and unified. This allows a single application to:

- Serve web pages for humans who interact through a browser.
- Provide data endpoints that consume JSON.